

TFP: Temporally Conditioned Memory-Fusion Policies for Visuomotor Learning

Yushen Liang^{1,†}, Yue Peng^{1,†}, Baosheng Jin^{1,†}, Tianluo Zhang¹,
Xinyu Zhang², Shuyi Zhou, Zhuoran Chen¹, Xinqi Liu¹, Shenji Wan¹

¹NYU Shanghai, Shanghai, China

²University of Electronic Science and Technology of China, Chengdu, China

[†]Equal contribution

🔗 Code: github.com/Mirage415/TFP-Temporally-conditioned-Memory-Fusion-Policies-for-Visuomotor-Learning

Abstract—Vision–Language–Action (VLA) policies such as $\pi_{0.5}$ and OpenVLA perform well on many manipulation tasks, but they are often reactive: the next action is predicted from the current observation, instruction, and proprioceptive state. This assumption breaks down in stage-dependent manipulation, where visually similar states may require different actions depending on latent task progress and previous interaction outcomes. We argue that such tasks require not only memory, but dynamics-aware belief updates: the policy should preserve task progress during stable or occluded phases and revise its belief near contact, release, or subgoal transitions. We introduce Temporally Conditioned Memory-Fusion Policies (TFP), a lightweight memory-action framework for VLA backbones. TFP maintains an episode-local task-progress belief with Liquid Time-Constant dynamics and injects the updated belief directly into the flow-matching action decoder through adaptive modulation. This lets temporally accumulated context shape the generated action chunk, rather than serving only as passive history context. With a 3.3B-parameter model, TFP improves the average success rate from 96.9% to 98.75% on LIBERO and from 91.4% to 93.77% on LIBERO-plus. On the memory-focused MIKASA ShellGame-Touch diagnostic, TFP achieves success up to 75.0%. Mechanistic analyses show that write-gain changes near manipulation events are about $6\times$ larger than far non-event phases, and hidden-state interventions show that the belief causally modulates generated action chunks. These results suggest that compact, event-sensitive memory dynamics can improve VLA policies under occlusion, visual perturbation, and stage-dependent task structure.

Index Terms—Imitation Learning, Manipulation, Memory-Augmented VLA

I. INTRODUCTION

Vision–Language–Action (VLA) models, such as $\pi_{0.5}$, OpenVLA, and Octo, have achieved strong performance in robotic manipulation by mapping language instructions and multimodal observations directly to actions [2, 3, 8, 16, 22]. However, many VLA policies remain largely reactive: each action is predicted from the current observation, instruction, and proprioceptive state. This is insufficient for stage-dependent manipulation, where the same visible scene may require different actions depending on latent task progress and previous interaction outcomes. For example, in object swapping, a visually similar state may correspond to moving the first object to a buffer, moving the second object into the first object’s original location, or terminating the task.

This motivates adding memory, or task belief, to VLA policies. We use “belief” to denote a learned latent memory

state that summarizes hidden task progress relevant to future actions. However, robotic manipulation requires more than storing past observations. A useful belief must also decide *when* to change. During stable transport or occlusion, the policy should preserve task-progress information despite ambiguous visual evidence; near contact, release, or subgoal completion, it should rapidly incorporate new evidence. Thus, the key question is not only whether a VLA has memory, but whether its memory update follows the event structure and temporal dynamics of manipulation.

Existing memory-aware robot policies mainly either retrieve from history buffers or maintain recurrent latent states [27, 26, 30, 33, 18]. History-based methods preserve rich episodic detail but do not explicitly form a compact task-progress state; recurrent methods provide such a state, but updates are typically indexed by policy updates rather than manipulation events and elapsed physical time.

We argue that chunked visuomotor control calls for a dynamics-aware belief update. A practical controller may requery earlier when a chunk becomes unreliable, unstable, or requires correction after contact. Thus, the interval between policy queries is a physical-time variable, not merely a discrete step index. Liquid Time-Constant (LTC) networks provide a natural mechanism for such belief tracking. Their hidden states evolve through input-dependent time constants, allowing the model to retain information when task context should persist and to update more rapidly when new evidence appears [13, 14]. This continuous-time inductive bias is well matched to memory-dependent manipulation, where completed subgoals, prior contacts, and interaction outcomes may remain relevant across multiple action chunks.

We introduce **Temporally Conditioned Memory-Fusion Policies (TFP)**, a belief-conditioned VLA framework for robotic manipulation. TFP maintains an episode-local latent belief with LTC dynamics: the previous hidden state carries retained task context, the current visual and proprioceptive observation proposes an update, and elapsed time controls how strongly the belief is retained or revised. The updated belief is projected into the action-head conditioning space and injected through AdaLN-style modulation, allowing temporal context to directly shape the flow-matching action distribution rather than serve only as passive historical context.

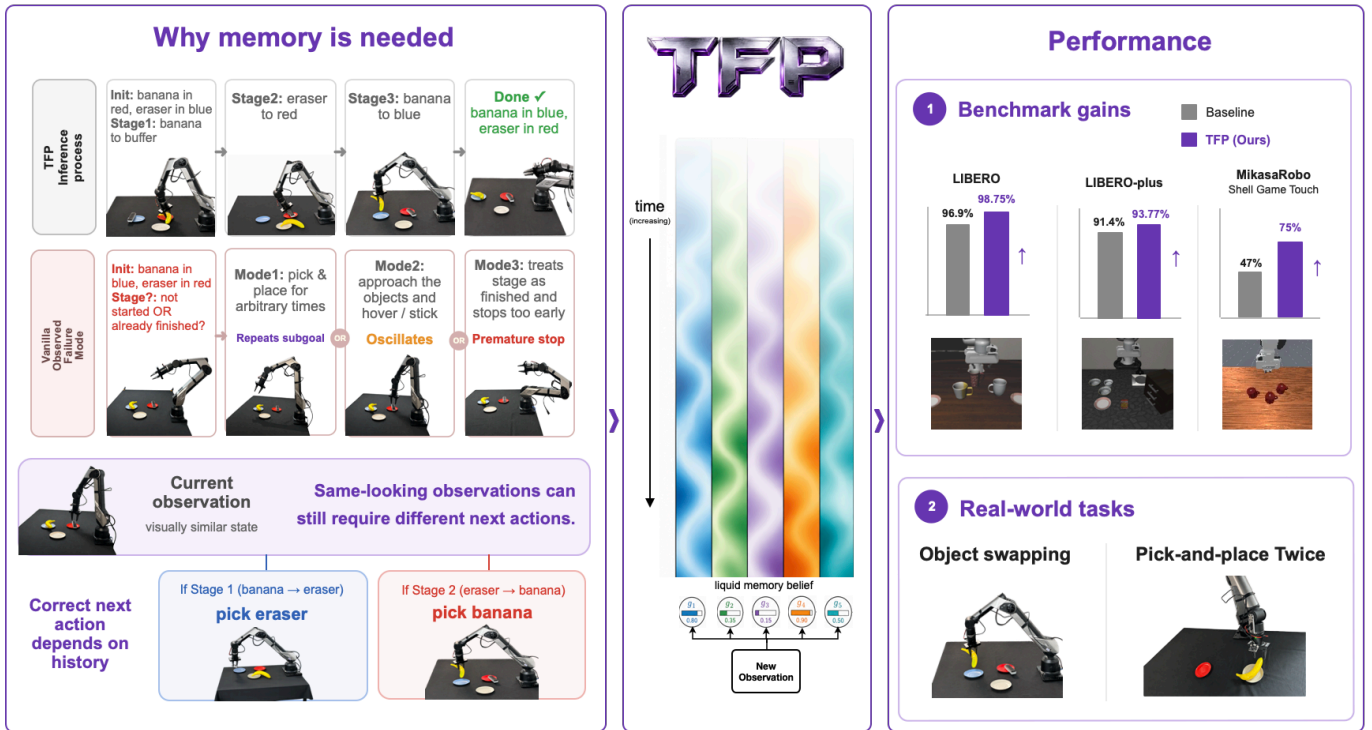


Fig. 1: Overview illustrating the motivation for memory-conditioned visuomotor control.

We evaluate TFP on simulation benchmarks and real-world memory-dependent manipulation tasks. TFP consistently improves over reactive, history-based, and latent-state baselines. Mechanistic analyses further show that the learned belief stabilizes task-progress representations and causally modulates generated action chunks. Our contributions are:

- We formulate stage-dependent manipulation as dynamics-aware belief tracking, where the correct action depends not only on latent task progress and interaction history, but also on when the internal belief should be revised or preserved.
- We propose TFP, a belief-conditioned VLA framework that maintains a continuous-time latent belief using LTC dynamics and injects this belief directly into flow-matching action generation. We further introduce Episode-Aware Temporal Batching (EATB) to train recurrent belief states efficiently while preserving episode-local hidden-state continuity.
- We show that TFP improves memory-dependent control in simulation and on a real Galaxea A1 robot, and provide mechanistic analyses showing that the learned belief stabilizes task-progress representations and causally modulates generated action chunks.

II. RELATED WORK

a) Memory in vision-language-action policies.: Recent VLA policies such as RT-2, OpenVLA, Octo, and π_0 map language, visual observations, and robot states directly to actions [3, 16, 22, 2]. Since many manipulation tasks are partially observable, several works augment such policies with

history or memory. One class stores past observations, actions, or semantic summaries and retrieves or attends to them at decision time, as in HAMLET, MemoryVLA, and Causal Diffusion Policy [17, 27, 20]. These methods improve access to past context, but memory is mainly used as a retrieval source rather than as an explicit state of task progress. A second class maintains a compact latent state. AVA-VLA uses a recurrent belief state to modulate visual processing, and ReMem-VLA propagates recurrent memory queries across frames and action chunks [33, 18]. These methods provide explicit memory, but their updates are typically tied to frames, policy steps, or chunks. In chunked receding-horizon control, however, the interval between policy queries can vary with execution progress, contact, or instability. The memory update should therefore depend on physical elapsed time, not only on a discrete step index.

b) Continuous-time belief for chunked control.: Liquid Time-Constant networks and closed-form continuous-time networks model hidden-state evolution with input-dependent time constants [13, 14]. TFP uses this continuous-time mechanism as a task-belief filter for chunked VLA control. The belief retains slow task-progress information, updates when new visual and proprioceptive evidence arrives, and directly modulates the flow-matching action decoder through AdaLN-style conditioning. Thus, memory in TFP is not only retrieved context or an auxiliary perception signal, but a compact physical-time belief state that directly shapes action generation. More discussion on broader related work can be found in Appendix A.

III. METHOD

TFP augments a chunked VLA policy with an episode-local latent belief over task progress and interaction history. At each policy query, the current visual and proprioceptive observations update this belief through Liquid Time-Constant (LTC) dynamics, and the updated belief directly conditions the flow-matching action head through AdaLN-style modulation. This couples memory and action generation: temporal context is not stored as a passive history buffer, but as a persistent state that modulates the next generated action chunk.

A. Problem Setting and Notation

At policy query t , the agent receives a visual observation $\mathcal{I}_t$, robot state $\mathbf{s}_t \in \mathbb{R}^{d_s}$, elapsed time $\Delta t_t \in \mathbb{R}_{\geq 0}$ since the previous query, language instruction ℓ , and previous belief $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$. A vision encoder maps $\mathcal{I}_t$ to visual tokens

$$\mathbf{V}_t = [\mathbf{v}_{t,1}, \dots, \mathbf{v}_{t,N}] \in \mathbb{R}^{N \times d_v}. \quad (1)$$

The policy updates the latent belief $\mathbf{h}_t$ and predicts an action chunk conditioned on the current observation, robot state, instruction, and belief.

B. Continuous-Time Belief Update

We first form a compact observation representation $\mathbf{x}_t = [\phi_{\text{vision}}(\mathbf{V}_t); \phi_{\text{state}}(\mathbf{s}_t)]$, where $\phi_{\text{vision}}(\cdot)$ pools or projects visual tokens and $\phi_{\text{state}}(\cdot)$ embeds the robot state. TFP updates the latent belief with an LTC-style recurrence:

$$\begin{aligned} \hat{\mathbf{h}}_t &= \tanh(W_h[\mathbf{x}_t; \mathbf{h}_{t-1}] + b_h), \\ \tau_t &= \text{softplus}(W_\tau[\mathbf{x}_t; \mathbf{h}_{t-1}] + b_\tau) + \epsilon. \end{aligned} \quad (2)$$

$$\begin{aligned} \mathbf{k}_t &= \exp(-\Delta t_t / \tau_t), \\ \mathbf{h}_t &= \mathbf{k}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{k}_t) \odot \hat{\mathbf{h}}_t. \end{aligned} \quad (3)$$

Here $\hat{\mathbf{h}}_t$ is the candidate belief induced by the current observation and previous belief, $\tau_t \in \mathbb{R}_{>0}^{d_h}$ is a vector of input-dependent time constants, and $\mathbf{k}_t$ controls how much of the previous belief is retained over the elapsed interval Δt_t . Equivalently, with write gain $\mathbf{g}_t = 1 - \mathbf{k}_t$, the update can be written as

$$\mathbf{h}_t - \mathbf{h}_{t-1} = \mathbf{g}_t \odot (\hat{\mathbf{h}}_t - \mathbf{h}_{t-1}). \quad (4)$$

Thus, memory changes only when the candidate belief differs from the retained belief and the elapsed-time-dependent write gain permits the update. This is useful for manipulation: slow channels can preserve task context across action chunks, while faster channels can absorb new perceptual evidence when the scene changes. We use *belief* in a functional sense: $\mathbf{h}_t$ is a learned latent state summarizing task progress and interaction history, rather than an explicit Bayesian posterior or a raw buffer of past observations.

This shows that memory changes only when the current candidate belief differs from the retained belief and the write gain allows new evidence to be incorporated. In manipulation, this is useful because observations can be noisy, partially occluded, or locally ambiguous. Once the policy has formed

a task-conditioned belief, later observations may produce candidate beliefs that are already consistent with the retained state, yielding small updates without making the memory uninformative.

The vector-valued time constant τ_t allows different latent channels to operate at different effective time scales. We do not assume that individual coordinates correspond to human-defined events; rather, task progress, object state, contact evidence, and occlusion-related information may be represented as distributed latent patterns. Some subspaces can therefore behave as high-retention channels for slow task context, while others can update more quickly with local perceptual evidence.

For each latent dimension j , the update unrolls as

$$h_{t,j} = \left(\prod_{r=1}^t k_{r,j} \right) h_{0,j} + \sum_{i=1}^t (1 - k_{i,j}) \left(\prod_{r=i+1}^t k_{r,j} \right) \hat{h}_{i,j}. \quad (5)$$

Thus, the memory forms a path-dependent mixture of past candidate beliefs: a candidate contributes only if it is written when observed and retained by later updates. This explains the write-then-stabilize behavior observed in our diagnostics. A useful memory trajectory need not keep changing throughout a rollout; after informative observations establish the task belief, high retention can preserve it through occlusion, contact noise, or visually ambiguous stages.

C. Belief-Conditioned Action Generation

TFP injects the updated belief into the flow-matching action decoder through adaptive normalization rather than explicit memory-token cross-attention. Given $\mathbf{h}_t$, we project it into the decoder conditioning space:

$$\mathbf{m}_t = W_m \mathbf{h}_t + \mathbf{b}_m \in \mathbb{R}^{d_c}. \quad (6)$$

The flow-matching timestep is embedded as

$$\mathbf{e}_\tau = \text{PE}(\tau), \quad \mathbf{z}_\tau = W_2 \sigma(W_1 \mathbf{e}_\tau + \mathbf{b}_1) + \mathbf{b}_2, \quad (7)$$

where $\sigma(\cdot)$ is a swish nonlinearity. We combine timestep and belief conditioning by

$$\mathbf{c}_t = \mathbf{z}_\tau + \mathbf{m}_t, \quad \hat{\mathbf{x}}^{(\ell)} = \text{AdaLN}(\mathbf{x}^{(\ell)}, \mathbf{c}_t), \quad (8)$$

where $\mathbf{x}^{(\ell)}$ is the hidden state of decoder layer ℓ . AdaLN predicts feature-wise affine modulation parameters from $\mathbf{c}_t$ and applies them to normalized decoder activations [23, 24]. The resulting action head predicts the memory-conditioned flow velocity

$$v_\theta(a_\tau, \tau \mid \mathcal{I}_t, \mathbf{s}_t, \ell, \mathbf{h}_t). \quad (9)$$

This interface makes the belief directly modulate the generated action chunk, instead of requiring the decoder to retrieve and interpret a separate memory token sequence. Consequently, visually similar observations can produce different actions when their retained task beliefs differ. Additional discussion of this adaptive-conditioning interface is given in Appendix B-A.

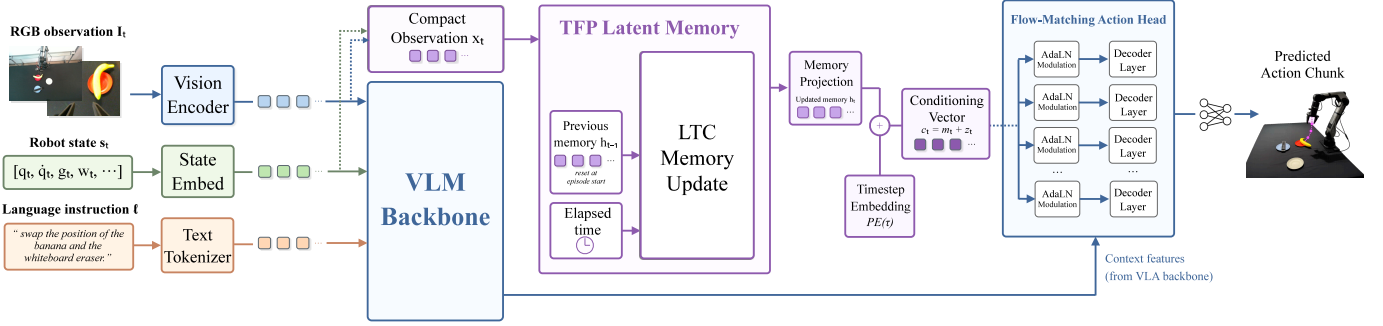


Fig. 2: TFP maintains a continuous-time latent belief and injects it into the action decoder through adaptive modulation.

D. Training with Episode-Aware Temporal Batching

Training TFP requires preserving hidden-state continuity within each demonstration episode. Randomly shuffling individual chunks breaks this continuity, while full-episode backpropagation is memory-intensive for large VLA backbones. We therefore introduce *Episode-Aware Temporal Batching* (EATB), which trains on temporally contiguous chunks from multiple episodes in parallel while maintaining a separate hidden state for each active episode.

For each episode e , EATB keeps an episode-local hidden state $\mathbf{h}_t^{(e)} = f_\theta(\mathbf{h}_{t-1}^{(e)}, o_t^{(e)}, \Delta t_t^{(e)})$ and resets it only at episode boundaries. At each training step, we sample B active episodes and unroll each for K consecutive chunks. If episode e_b starts from chunk index τ_b , its stored hidden state is retrieved as $\mathbf{h}_{b,0} = \tilde{\mathbf{h}}_{\tau_b}^{(e_b)}$ and updated through the segment. We optimize the average flow-matching imitation loss over the resulting BK chunks,

$$\mathcal{L}_s = \frac{1}{BK} \sum_{b=1}^B \sum_{k=1}^K \ell_{\text{flow}}^{(b,k)}. \quad (10)$$

After the parameter update, the final hidden state is written back with gradient truncation:

$$\tilde{\mathbf{h}}_{\tau_b+K}^{(e_b)} \leftarrow \text{stopgrad}(\mathbf{h}_{b,K}). \quad (11)$$

Thus, hidden states persist numerically across training segments, while gradients are truncated at segment boundaries. EATB preserves long-horizon forward memory without the cost of full-episode backpropagation. Further implementation details are described in Appendix B.

E. Adaptive Receding-Horizon Execution

To evaluate TFP under irregular policy-query intervals, we use an inference-time adaptive receding-horizon executor. The policy always predicts an action chunk of horizon H , but the executor selects a prefix length $E_t \in [E_{\min}, H]$ to execute before the next policy query. The next memory update receives the actual elapsed interval

$$\Delta t_{t+1} = E_t \delta t_{\text{ctrl}}. \quad (12)$$

The executor executes the full horizon by default and shortens the prefix only when the predicted chunk contains a gripper-transition boundary or a sustained high-risk region. The per-step chunk risk is

$$R_{t,r} = \lambda_j J_{t,r} + \lambda_b B_{t,r} + \lambda_c C_{t,r}, \quad (13)$$

where $J_{t,r}$ measures action jerk, $B_{t,r}$ indicates gripper-transition boundaries, and $C_{t,r}$ measures continuity with the previously executed action. This changes only the inference-time execution schedule; the policy network, predicted action horizon, and model weights are unchanged.

IV. COMPARISON TO RECURRENT AND STATE-SPACE BASELINES

A. Relation to GRUs

A standard GRU updates its hidden state as

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t, \quad z_t = \sigma(W_z[x_t, h_{t-1}] + b_z). \quad (14)$$

This update resembles the interpolation form used by TFP, but its retention gate is indexed by recurrent steps rather than physical time. In particular, when elapsed time is not provided as an input, two updates with the same observation and previous hidden state necessarily produce the same gate, even if they occur after different physical intervals.

This differs from continuous-time belief dynamics. For example, in the absence of new evidence, exponential decay evolves as

$$h_t = \exp(-\lambda \Delta t_t) h_{t-1}, \quad \lambda > 0, \quad (15)$$

and therefore produces different retained states for different values of Δt_t . A standard step-indexed GRU without access to Δt_t does not enforce this elapsed-time dependence. In contrast, TFP explicitly parameterizes its retention as

$$k_t = \exp(-\Delta t_t / \tau_t), \quad (16)$$

so the belief update remains calibrated to the physical interval between policy queries.

This distinction does not imply that GRUs or LSTMs are incapable of representing temporal history. Rather, elapsed-time consistency must be learned from additional timing inputs

or introduced through architectural constraints, whereas it is built directly into the TFP update.

B. Relation to Fixed-Decay Memory

A fixed-decay memory updates its state according to

$$h_t = \exp(-\Delta t_t / \tau_0) \odot h_{t-1} + (1 - \exp(-\Delta t_t / \tau_0)) \odot \hat{h}_t, \quad (17)$$

where τ_0 is a constant time scale. This model is a special case of the TFP update obtained by setting $\tau_t = \tau_0$. TFP instead predicts

$$\tau_t = \tau_\theta(x_t, h_{t-1}), \quad (18)$$

allowing the effective retention time to depend on both the current evidence and the retained belief. Consequently, TFP preserves the elapsed-time structure of exponential retention while adapting how quickly individual memory channels update across different task states and interaction events.

C. Relation to State-Space Models

Linear state-space models provide efficient long-horizon sequence modeling through structured linear recurrences [12, 11]. Continuous-time SSMs can also implement elapsed-time-dependent transitions, for example

$$h_t = \exp(A\Delta t_t)h_{t-1} + B(\Delta t_t)x_t.$$

Therefore, we do not claim that LTC is more time-consistent than all SSMs. Instead, the distinction is functional: TFP uses LTC as a nonlinear, input-dependent belief fuser whose state directly conditions a generative action decoder. This makes the memory update both time-calibrated and action-relevant for chunked manipulation policies.

D. Computational Overhead

The LTC update costs $\mathcal{O}(d_h(d_x + d_h))$, where d_x is the compact observation dimension and d_h is the memory dimension. The memory projection costs $\mathcal{O}(d_h d_c)$, and AdaLN adds only lightweight affine modulation in the action decoder. These costs are small relative to the vision-language backbone and transformer-based action decoder; the main computational cost of TFP comes from recurrent training over multiple contiguous chunks.

V. EXPERIMENTS

Evaluation Questions. Our experiments ask whether TFP (i) improves standard and robustness-oriented manipulation benchmarks, (ii) improves real-world stage-dependent manipulation where the current observation alone is insufficient, (iii) encodes action-relevant task progress that causally affects action generation, and (iv) benefits from continuous-time LTC dynamics and real elapsed-time conditioning under irregular control intervals.

a) Experimental Setup. We evaluate TFP with $\pi_{0.5}$ as the primary VLA backbone on LIBERO, LIBERO-plus, MIKASA-Robo, and two Galaxea A1 real-world tasks: object swapping and counting pick-and-place. For our controlled same-backbone experiments, all reproduced variants use the same demonstrations, action horizon, optimizer, and rollout protocol. Published baselines are reported as literature references when available. Across experiments, TFP receives a language prompt, two RGB observations, and proprioceptive state at each policy query, and maintains an episode-local 256-dimensional memory state h_t . For recurrent methods, EATB is applied during training, memory states are episode-local and reset only at episode boundaries during inference.

For comparisons to published baselines, we report their original fixed-horizon numbers when available and mark them as literature references. For our controlled same-backbone ablations, including recurrent variants, TFP w/o Δt , and TFP, we use the same adaptive receding-horizon executor. Thus, policy queries occur after variable executed prefixes, and all controlled memory variants are evaluated under the same irregular-query schedule. The only difference between TFP and TFP w/o Δt is whether the memory update receives the measured elapsed interval or a constant step interval. The real-world setup uses a single-arm Galaxea A1 platform with a parallel gripper, a wrist-mounted fisheye camera, and a static Intel RealSense D435 camera. Predicted action chunks are executed as joint-angle commands through ROS1 Noetic.

Statistical reliability for the reproduced LIBERO and real-world results is reported in Appendix E.

A. Main Benchmark Results

We organize our evaluation around different benchmark purposes. LIBERO and LIBERO-plus measure online rollout performance and general manipulation quality, including long-horizon and multi-stage tasks. This is important because real-world manipulation is not always explicitly memory-dependent; a memory-augmented VLA should improve memory-sensitive behavior without sacrificing general task execution. Tables I summarizes the results on simulation benchmarks.

We then use memory-focused benchmarks and real-world experiments as sanity checks for whether the proposed memory state is used. MIKASA-Robo ShellGameTouch isolates belief-state filtering under occlusion in simulation, while our real-world A1 experiments evaluate memory-dependent manipulation on hardware. Thus, LIBERO benchmark tests generality and rollout quality, whereas MIKASA-Robo and real-world A1 experiments test the intended memory mechanism, and LIBERO-plus further evaluates zero-shot generality under occlusion and visual perturbations, where the belief state is expected to stabilize action generation when the current observation is incomplete or unreliable. Ablations with recurrent, SSM-based variants are reported in Sec. V-C.

LIBERO. On LIBERO, TFP improves the average success rate from 96.9% for the reactive $\pi_{0.5}$ baseline to 98.75%. The improvement is most visible on the *long* split, where

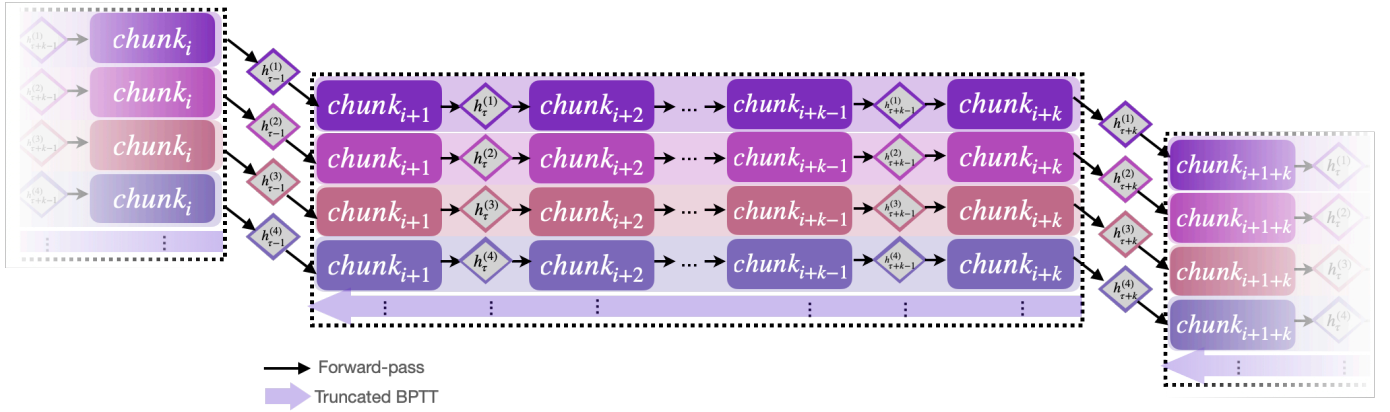


Fig. 3: Episode-Aware Temporal Batching trains on contiguous chunks while carrying episode-local hidden states across truncated segments.

TABLE I: Success rate (%) on LIBERO, LIBERO-plus, and MIKASA-Robo ShellGameTouch. TFP obtains the best average success rate on both LIBERO and LIBERO-plus while remaining in the 3.3B-parameter-only regime. Numbers for prior methods are reported results from published work and are not necessarily reproduced under our training pipeline. We use ShellGameTouch as a diagnostic rather than a state-of-the-art comparison.

Policy	LIBERO					LIBERO-plus				MIKASA-Robo ShellGameTouch
	Spatial	Object	Goal	Long-10	Avg.	Noise	Light	Backgr.	Avg.	
$\pi_{0.5}$ (3.3B)	98.8	98.0	98.2	92.4	96.85	85.2	93.9	95.1	91.4	—
π_0 (3.3B)	94.2	96.4	96.8	87.6	93.8	79.0	85.0	81.4	81.8	33
OpenVLA (7B)	84.7	88.4	79.2	53.7	76.5	15.2	8.1	34.8	19.37	12
OpenVLA-OFT (7.3B)	97.6	98.4	97.9	94.5	97.1	75.8	88.7	93.3	85.93	47
HAMLET (3B)	99.0	100.0	99.2	92.2	97.6	—	—	—	—	—
AVA-VLA [33] (7.3B)	97.4	99.4	97.4	97.6	98.0	78.0	95.5	88.9	87.47	—
MemoryVLA (7.3B)	98.4	98.4	96.4	93.4	96.65	—	—	—	—	88
TFP (ours) (3.3B)	99.6	99.0	99.4	97.0	98.75	88.5	96.1	96.7	93.77	75

TFP reaches 97.0% compared with 92.4% for $\pi_{0.5}$. This supports our hypothesis that maintaining latent task progress is especially beneficial for long-horizon manipulation, where the current observation alone may be insufficient for selecting the correct action.

LIBERO-plus. On LIBERO-plus, TFP achieves the best average robustness score of 93.77%, outperforming $\pi_{0.5}$ by 2.37 percentage points. The gains are most notable under noise and lighting perturbations, where TFP improves from 85.2% to 88.5% and from 93.9% to 96.1%, respectively.

MIKASA-Robo. On MIKASA-Robo ShellGameTouch, TFP achieves a success rate of 75.0%, substantially improving over the reported OpenVLA-OFT reference result at 47.0%. ShellGameTouch isolates belief-state filtering under occlusion: after the ball is hidden by visually identical mugs, the policy must remember the hidden ball location and use that belief to touch the correct mug. Since the Touch variant minimizes additional pushing or grasping requirements, the improvement more directly reflects memory-conditioned action selection rather than low-level manipulation skill. We do not present this result as a state-of-the-art claim: MemoryVLA reports 88.0% on this task. Instead, we use ShellGameTouch as a diagnostic showing that TFP improves hidden-state reasoning

in a reactive VLA, while the remaining gap points to a complementary need for object-centric hidden-location binding; see Appendix D-B.

Real-world A1. Table II reports real-robot success counts and dominant failure types among unsuccessful trials.

TABLE II: Real-world Galaxea A1 results and failure analysis.

Task	Method	Success	Stage	Repeat	Target	Exec.
Object swap	$\pi_{0.5}$	3/20	✓	✓	○	—
	TFP	15/20	—	—	○	✓
Counting pick-place	$\pi_{0.5}$	8/20	✓	✓	—	○
	TFP	18/20	—	○	○	✓

Failure tags: Stage = wrong phase/forgotten progress; Repeat = completed subtask repeated; Target = wrong object/placement grounding; Exec. = low-level grasp, placement, or contact instability.

Table II shows that TFP primarily reduces stage-level memory failures: the reactive baseline often repeats completed subtasks or switches to the wrong stage, whereas TFP failures shift toward target grounding and low-level execution errors. This suggests that the learned belief improves task-progress tracking rather than merely improving low-level manipulation.

B. Event-Sensitive Belief Dynamics

a) *Irregular-query diagnostic.*: We next examine whether the LTC memory learns an event-sensitive belief

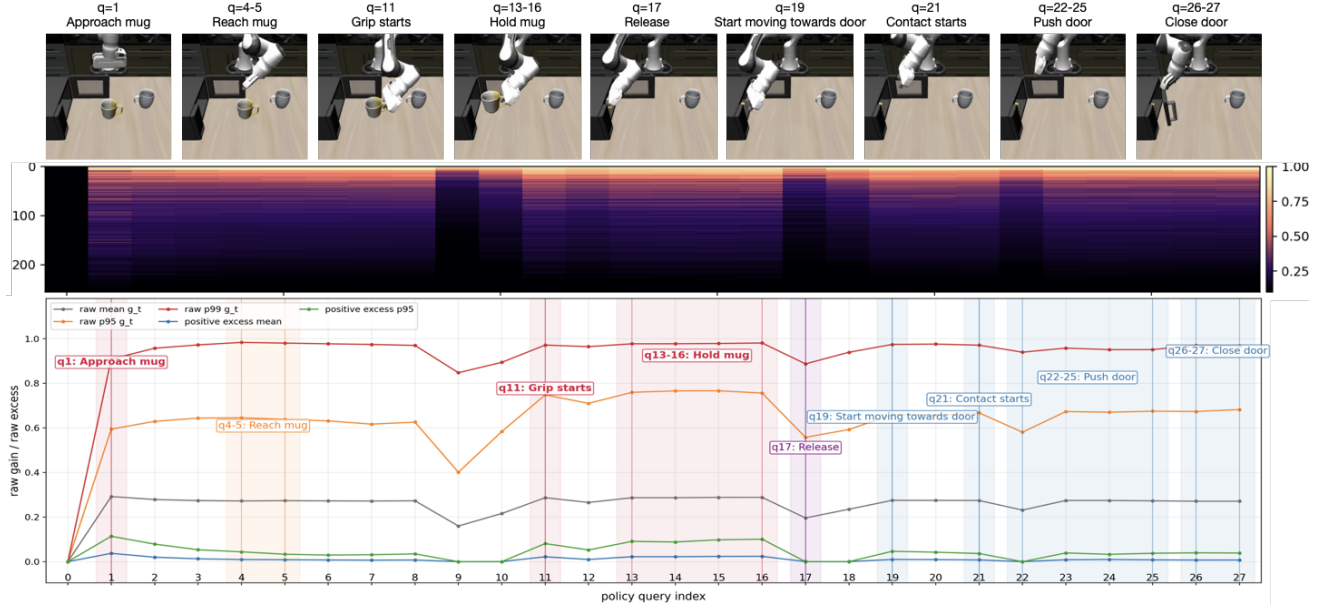


Fig. 4: LTC belief update gain g_t during a rollout. Frames, dimension-wise gate heatmaps, and summary statistics show gate changes around reaching, carrying, releasing, and pushing events.

update rather than a generic step-indexed recurrence. For this diagnostic, we run TFP with adaptive receding-horizon execution: the policy predicts an action chunk, but the executor may re-query after a variable executed prefix when the chunk crosses a gripper transition, contact-sensitive phase, or unstable region. The memory update therefore receives the elapsed interval between policy queries. This setting exposes the memory module to irregular physical time, so the learned update gain can reflect both manipulation events and elapsed time rather than a fixed recurrent step index.

At each policy query, we record the internal LTC variables during LIBERO rollouts: the time constant τ_t , retention gate k_t , update gain $g_t = 1 - k_t$, hidden state h_t , and candidate belief $\hat{h}_t$. Larger values of g_t indicate stronger writing from the current observation, whereas smaller values indicate stronger retention of the previous task belief. Thus, the temporal pattern of g_t directly reveals when the model chooses to revise or preserve its latent belief.

For interpretation only, we extract privileged simulator states such as gripper state, end-effector position, object poses, object height, and task success; these signals are never provided to the policy.

Figure 4 shows nonuniform LTC updates: some dimensions remain stable during transport, while others change near task transitions. To quantify this event sensitivity, we measure write-gain changes $D_t^g = \frac{1}{d_h} \sum_i |g_{t,i} - g_{t-1,i}|$. For each simulator-side event, we take the largest D_t^g within ± 6 queries and compare it with a far non-event baseline excluding ± 8 -query neighborhoods. Table III shows that gain-pattern changes near events are about $6\times$ larger than the far non-event baseline, indicating that the policy often adjusts its write/retain schedule slightly before the simulator-side event label.

TABLE III: Lagged alignment of LTC write-gain dynamics on LIBERO mug-in-microwave, using 224 events from 30 successful rollouts.

Metric	Center/base	Best ± 6 /base	95% CI	Lag
Mean dim. $ \Delta g_t $	$0.42\times$	$5.97\times$	$[4.61, 7.54]$	-1.64
Top-10% dim. $ \Delta g_t $	$0.48\times$	$5.90\times$	$[4.57, 7.44]$	-2.13
$ \Delta \bar{g}_t $	$0.34\times$	$6.24\times$	$[4.81, 7.88]$	-1.78
$\ \Delta g_t\ _2 / \sqrt{d_h}$	$0.35\times$	$6.14\times$	$[4.65, 7.88]$	-1.88

Following prior analyses of policy dynamics and task-relevant factors [32], we use event labels only post hoc: they are not provided to the policy, but reveal whether LTC update gains track manipulation changes.

b) Same Observation, Different Memory States.: Fixing the observation, robot state, and instruction while replacing only the LTC hidden state substantially changes the predicted 10-step action chunks: mean pairwise action distance is 0.0442, maximum 0.1303, while temporally adjacent hidden states have median distance 0.0108. Thus, the LTC belief actively conditions actions; see Appendix D-A.

C. Ablation Studies

We ablate whether TFP’s gains come merely from adding temporal state, and whether elapsed-time conditioning in the LTC update is necessary.

1) Memory Dynamics: We compare TFP with GRU and S4D memory variants under the same $\pi_{0.5}$ backbone, training data, action horizon, and LIBERO evaluation protocol. This tests whether gains come from generic recurrence or from continuous-time memory coupled to action-head conditioning.

GRU performs competitively on spatial tasks but degrades on object, goal, and long-horizon tasks; S4D improves over GRU but remains below TFP, especially on the long-horizon

TABLE IV: Controlled memory-dynamics ablations under the same adaptive receding-horizon executor. All same-backbone variants use the same demonstrations, action horizon, optimizer, rollout protocol, and irregular-query execution schedule. TFP w/o Δt uses the same LTC and AdaLN architecture as TFP but replaces the measured elapsed interval with a constant step size.

(a) LIBERO [19]						(b) LIBERO-plus [9]				
Method	Spatial	Object	Goal	Long	Avg.	Method	Noise	Light	Bg.	Avg.
$\pi_{0.5}$	98.8	98.0	98.2	92.4	96.9	$\pi_{0.5}$	85.2	93.9	95.1	91.4
$\pi_{0.5}$ + GRU	99.0	98.5	95.5	89.0	95.5	$\pi_{0.5}$ + GRU	85.9	91.5	96.1	91.2
$\pi_{0.5}$ + SSM (S4D)	98.5	98.5	97.0	92.5	96.625	$\pi_{0.5}$ + SSM (S4D)	82.6	95.0	92.3	90.0
TFP w/o Δt	98.5	97.5	97.5	94.0	96.9	TFP w/o Δt	82.5	93.8	94.1	90.1
TFP (ours)	99.6	99.0	99.4	97.0	98.75	TFP (ours)	88.5	96.1	96.7	93.8

split. Thus, gains are not from memory alone: the state must be temporally stable and directly coupled to action generation.

a) Elapsed-time conditioning. Because adaptive receding-horizon execution re-queries the policy after variable executed prefixes, the interval between memory updates is nonuniform. We isolate the role of measured elapsed time by comparing TFP with TFP w/o Δt , which keeps the same LTC update, AdaLN action conditioning, training data, and adaptive executor, but replaces the measured interval with a constant step size. The drop from TFP to TFP w/o Δt , especially on LIBERO Long and LIBERO-plus, shows that the gain is not only from adding a recurrent state or adaptive execution; the memory update benefits from receiving the actual elapsed time between policy queries.

On LIBERO-plus, GRU improves some perturbations but lowers average robustness relative to $\pi_{0.5}$, while TFP improves the average from 91.4% to 93.8%, mainly under noise and lighting. Memory must therefore stabilize action generation rather than merely add recurrent state.

D. Limitations

Although LTC and AdaLN add little inference overhead, recurrent full fine-tuning is costly because training must unroll across action chunks to preserve hidden-state continuity and credit assignment. EATB truncates gradients across contiguous chunks, but our largest LIBERO runs still used $K = 8$, batch size 128, over 100GB GPU memory, and about 80 hours on 4 H200 GPUs to reach imitation loss near 0.003. Our real-world evaluation is also limited to tabletop single-arm manipulation, leaving mobile manipulators, humanoids, and dexterous hands for future work.

VI. CONCLUSION

We presented Temporally Conditioned Memory-Fusion Policies (TFP), a framework for extending reactive VLA policies with an explicit task-progress belief for history-dependent manipulation. TFP maintains this belief with Liquid Time-Constant dynamics and injects it directly into the flow-matching action decoder through AdaLN-style conditioning, allowing the learned belief to shape action generation rather than serve as passive history context. Across simulation and real-world tasks, TFP improves performance most clearly when the current observation alone is insufficient, including long-horizon, visually perturbed, occluded, and stage-dependent manipulation settings. Mechanistic analyses show

that the learned write-gain dynamics change sharply near manipulation-event boundaries, while hidden-memory interventions show that changing only the belief can alter the generated action chunk and end-effector trajectory. These results suggest that memory in VLA policies should act as an event-sensitive action-conditioning state: retaining task progress during stable or ambiguous phases, and revising the belief when task-relevant evidence changes. Future work should combine such dynamics-aware memory with stronger object-centric grounding, spatial generalization, and compositional planning, while developing more efficient recurrent fine-tuning schemes.

REFERENCES

- [1] Inci M Baytas, Cao Xiao, Xi Zhang, Fei Wang, Anil K Jain, and Jiayu Zhou. Patient subtyping via time-aware lstm networks. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 65–74, 2017.
- [2] Kevin Black et al. π_0 : A flow-matching policy for generalist robot manipulation. *arXiv preprint arXiv:2410.24164*, 2024. Physical Intelligence.
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [4] Zhengping Che, Sanjay Purushotham, Kyunghyun Cho, David Sontag, and Yan Liu. Recurrent neural networks for multivariate time series with missing values. *Scientific reports*, 8(1):6085, 2018.
- [5] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- [6] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023.
- [7] Kyunghyun Cho, Bart Van Merriënboer, Çağlar Gulçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1724–1734, 2014.
- [8] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. PaLM-E: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.
- [9] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Ji Li, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [10] Ken-ichi Funahashi and Yuichi Nakamura. Approximation of dynamical systems by continuous time recurrent neural networks. *Neural networks*, 6(6):801–806, 1993.
- [11] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [12] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.
- [13] Ramin Hasani, Mathias Lechner, Alexander Amini, Daniela Rus, and Radu Grosu. Liquid time-constant networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 7657–7666, 2021.
- [14] Ramin Hasani, Mathias Lechner, Alexander Amini, Hongxu Sun, Arash Mehrjou, Radu Grosu, and Daniela Rus. Closed-form continuous-time neural networks. *Nature Machine Intelligence*, 4(11):992–1003, 2022. doi: 10.1038/s42256-022-00556-7.
- [15] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [16] Moo Jin Kim et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [17] Myungkyu Koo, Daewon Choi, Taeyoung Kim, Kyungmin Lee, Changyeon Kim, Younggyo Seo, and Jinwoo Shin. Hamlet: Switch your vision-language-action model into a history-aware policy. *arXiv preprint arXiv:2510.00695*, 2025.
- [18] Hang Li, Fengyi Shen, Dong Chen, Liudi Yang, Xudong Wang, Jinkui Shi, Zhenshan Bing, Ziyuan Liu, and Alois Knoll. Remem-vla: Empowering vision-language-action model with memory via dual-level recurrent queries. *arXiv preprint arXiv:2603.12942*, 2026. doi: 10.48550/arXiv.2603.12942.
- [19] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.
- [20] Jiahua Ma et al. CDP: Towards robust autoregressive visuomotor policy learning via causal diffusion. *arXiv preprint arXiv:2506.14769*, 2025.
- [21] Daniel Neil, Michael Pfeiffer, and Shih-Chii Liu. Phased lstm: Accelerating recurrent network training for long or event-based sequences. *Advances in neural information processing systems*, 29, 2016.
- [22] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Charles Xu, Jianlan Luo, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [23] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4195–4205, October 2023.
- [24] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [25] Yulia Rubanova, Ricky T. Q. Chen, and David Duvenaud. Latent ordinary differential equations for irregularly-sampled time series. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [26] Rutav Shah, Rajat Kumar Jenamani, Xiaohan Zhang, Lingfeng Sun, Roberto Martín-Martín, Yuke Zhu, Deva

- Ramanan, and Karl Schmeckpeper. Scaling short-term memory of visuomotor policies for long-horizon tasks, 2026. URL <https://openreview.net/forum?id=5SMNtmJFGa>. Submitted to ICLR 2026.
- [27] Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2508.19236*, 2025.
 - [28] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, pages 785–799. PMLR, 2023.
 - [29] Qingxiong Tan, Mang Ye, Baoyao Yang, Siqu Liu, Andy Jinhua Ma, Terry Cheuk-Fung Yip, Grace Lai-Hung Wong, and PongChi Yuen. Data-gru: Dual-attention time-aware gated recurrent unit for irregular multivariate time series. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 930–937, 2020.
 - [30] Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z. Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, Karan Dhabalia, Michael Equi, Quan Vuong, Jost Tobias Springenberg, Sergey Levine, Chelsea Finn, and Danny Driess. Mem: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026. doi: 10.48550/arXiv.2603.03596.
 - [31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
 - [32] Tsun-Hsuan Wang, Wei Xiao, Tim Seyde, Ramin Hasani, and Daniela Rus. Measuring interpretability of neural policies of robots with disentangled representation. In *7th Annual Conference on Robot Learning*, 2023. URL <https://openreview.net/forum?id=6kSohKYYTn0>.
 - [33] Lei Xiao, Jifeng Li, Juntao Gao, Feiyang Ye, Yan Jin, Jingjing Qian, Jing Zhang, Yong Wu, and Xiaoyuan Yu. AVA-VLA: Improving vision-language-action models with active visual attention. *arXiv preprint arXiv:2511.18960*, 2025. doi: 10.48550/arXiv.2511.18960.
 - [34] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023.

The appendix provides additional material omitted from the main paper for space. Appendix A gives a broader discussion of related work. Appendix B describes implementation and execution details. Appendix C formalizes the continuous-time belief interpretation of LTC memory. Appendix IV compares TFP with recurrent, fixed-decay, and state-space alternatives. Appendix D provides additional experimental details, including hidden-state interventions and MIKASA-Robo failure analysis. Appendix E reports statistical reliability results for simulation and real-world rollouts.

APPENDIX A ADDITIONAL RELATED WORK

a) Vision-language-conditioned robot policies.: Robot manipulation has increasingly moved from task-specific behavior cloning toward vision-language-conditioned policies that map images, language instructions, and robot states to actions. RT-2, OpenVLA, Octo, and π_0 extend this direction to generalist VLA policies trained with large-scale robot and vision-language data [3, 16, 22, 2]. Related architectures such as PerAct further show the benefit of structured visual representations for language-conditioned manipulation [28]. TFP builds on this family of policies, but targets a different failure mode: in stage-dependent manipulation, the current observation and instruction may not uniquely determine the correct action without an internal belief over task progress, hidden object state, or previous interaction outcomes.

b) Chunked generative action policies.: Recent imitation policies often predict action chunks rather than single-step actions. ACT learns sequence-level action prediction for fine-grained manipulation, Diffusion Policy represents visuomotor control as conditional denoising over action trajectories, and π_0 uses flow matching for generalist action generation [34, 6, 2]. Action chunking improves temporal consistency and reduces policy-query frequency, but it also creates a memory problem: after executing part of a chunk, the policy must decide what task-progress information should persist to the next query. TFP addresses this by maintaining a latent belief across policy queries and injecting it directly into the action decoder.

c) Temporal and continuous-time memory models.: Temporal context can be modeled with recurrent networks, self-attention, state-space models, and continuous-time recurrent dynamics. RNNs, LSTMs, and GRUs provide compact recurrent states for history aggregation, with LSTMs designed for long time-lag dependencies and GRUs offering a gated recurrence [15, 7]. Time-aware variants further incorporate elapsed time through decay mechanisms or temporal gates, including GRU-D, T-LSTM, DATA-GRU, and Phased LSTM [4, 1, 29, 21]. These models are natural baselines for history-dependent control, but in chunked visuomotor policies they still require sequential unrolling over policy queries. Because each recurrent step may involve a large VLA backbone, an action-generation decoder, and a full action-horizon imitation loss, repeatedly calling the policy across many memory steps

can substantially increase training cost and limit temporal parallelism.

Self-attention provides parallel content-based access to history, but using past observations and action chunks as an ever-growing token set increases context length and does not by itself impose a physical-time belief dynamics [31]. Structured state-space models such as S4 and selective SSMs such as Mamba offer efficient long-sequence modeling with linear or near-linear scaling [12, 11]. These methods are strong sequence encoders, but our setting requires more than long-context compression: the policy must maintain an online task-progress belief whose retention is calibrated by elapsed time and whose state directly conditions action generation.

Continuous-time recurrent models provide another route to temporal memory. Classical continuous-time recurrent neural networks model hidden-state evolution with differential equations, while Neural ODEs, ODE-RNNs, and LatentODEs evolve hidden states continuously between irregular observations [10, 5, 25]. Liquid Time-Constant networks and closed-form continuous-time networks further parameterize input-dependent time constants for adaptive continuous-time sequence modeling [13, 14]. TFP builds on this continuous-time memory family, but uses LTC not as a standalone sequence model: we use it as a compact belief filter for chunked VLA control, with vector-valued, input-dependent time constants and closed-form elapsed-time retention. The resulting memory is then projected into the action-head conditioning space, so temporal belief directly modulates the generated action chunk instead of remaining a passive history representation.

d) Memory-augmented and history-aware visuomotor policies.: Several recent works augment robot policies with history or memory to address partial observability and long-horizon manipulation. HAMLET adapts pretrained VLAs into history-aware policies by introducing moment tokens and a lightweight memory module that aggregates historical context for action prediction [17]. This provides an efficient alternative to naively appending multiple past frames, but its memory primarily serves as a token-level history compression and retrieval interface. MemoryVLA introduces a cognition-memory-action framework, where perceptual and cognitive tokens form working memory, retrieve decision-relevant entries from a memory bank, and condition a diffusion action expert [27]. This design preserves richer historical details, but the temporal state is organized around memory storage, retrieval, and consolidation rather than an explicit elapsed-time-conditioned task dynamics. AVA-VLA reformulates VLA control from a POMDP perspective and uses a recurrent belief state to actively modulate visual token processing [33]. However, its memory is mainly used to guide perception, whereas the action-generation pathway is not explicitly conditioned by a continuous-time task-progress belief.

History-aware diffusion policies are also related. Causal Diffusion Policy conditions diffusion-based visuomotor control on historical action sequences through a causal transformer, improving temporal coherence and robustness under degraded observations [20]. Its focus is autoregressive action history

modeling rather than language-conditioned VLA control, and its own limitation notes that extremely long-horizon tasks requiring sustained planning remain unresolved. These methods show that exposing the policy to better, richer, or more retrievable history is valuable. TFP addresses a complementary problem: the policy should not only access history, but should maintain a compact latent belief whose evolution follows temporal-conditioned task dynamics.

Concretely, TFP compresses history into a fixed-size continuous-time belief state with input-dependent LTC retention and injects this belief directly into the generative action decoder through AdaLN-style modulation. Thus, memory is not only a retrieved context or an auxiliary visual-attention signal; it directly modulates the decoder activations and therefore the flow-matching velocity field over the predicted action chunk. This makes TFP better aligned with real manipulation dynamics, where the correct action depends on how task progress evolves over physical time, not merely on whether the model can attend to a more complete history.

APPENDIX B ADDITIONAL METHOD DETAILS

A. Memory-Conditioning Interface

TFP conditions the action decoder through adaptive modulation rather than explicit memory-token cross-attention. Cross-attention-based memory injection requires the decoder to retrieve and interpret a separate set of memory tokens. In contrast, adaptive conditioning compresses temporal context into a single modulation vector that directly shapes decoder activations. The timestep embedding captures where the denoising process is, while the memory condition captures what temporal context is relevant. Their additive fusion lets the decoder jointly adapt to diffusion phase and retained task belief without adding an extra attention pathway.

This choice reflects the role of LTC memory as a compact time-consistent belief state rather than a retrievable history buffer. We do not claim that cross-attention is inherently inferior; instead, AdaLN provides a simpler fixed-size interface for injecting belief into every action-decoder layer. A computational overhead discussion is provided in Appendix IV-D.

B. Episode-Aware Temporal Batching Details

For each episode e , EATB keeps an episode-local hidden state $\mathbf{h}_t^{(e)} = f_\theta(\mathbf{h}_{t-1}^{(e)}, o_t^{(e)}, \Delta t_t^{(e)})$, with $\mathbf{h}_0^{(e)} = \mathbf{0}$, and resets it only at episode boundaries. At training step s , we sample B active episodes and unroll each for K consecutive chunks. If episode e_b starts from chunk index τ_b , its stored hidden state is retrieved as $\mathbf{h}_{b,0} = \tilde{\mathbf{h}}_{\tau_b}^{(e_b)}$ and updated as

$$\mathbf{h}_{b,k} = f_\theta(\mathbf{h}_{b,k-1}, \mathbf{x}_{b,k}, \Delta t_{b,k}), \quad k = 1, \dots, K. \quad (19)$$

The optimization objective is

$$\mathcal{L}_s = \frac{1}{BK} \sum_{b=1}^B \sum_{k=1}^K \ell_{\text{flow}}^{(b,k)}. \quad (20)$$

After the parameter update, the final hidden state is written back with gradient truncation,

$$\tilde{\mathbf{h}}_{\tau_b+K}^{(e_b)} \leftarrow \text{stopgrad}(\mathbf{h}_{b,K}). \quad (21)$$

This preserves forward memory across training segments while bounding the backpropagation length.

APPENDIX C CONTINUOUS-TIME BELIEF INTERPRETATION

This section formalizes the connection between Liquid Time-Constant (LTC) memory and belief filtering in partially observable manipulation. We do not claim that LTCs are universally more expressive than all recurrent models. A sufficiently large recurrent network with access to elapsed time may approximate many belief-update functions. Instead, our claim is more specific: LTC provides a structural inductive bias that matches a class of continuous-time, multi-timescale belief dynamics. This class is particularly relevant for chunked robot manipulation, where policy queries may occur after variable execution intervals.

a) Claim.: For manipulation tasks that can be modeled as partially observable decision processes with latent task state, the desired memory should behave as a time-consistent belief state. LTC implements the exact discretization of a first-order continuous-time belief relaxation process, and its exponential retention gate is the unique continuous scalar retention rule satisfying elapsed-time consistency. Step-indexed recurrent baselines such as GRUs or LSTMs do not enforce this property unless elapsed time is explicitly provided and the gate is constrained to satisfy the same exponential semigroup structure.

A. Belief Filtering in a Continuous-Time POMDP

Let z_t denote the latent task state, which may include task phase, object state, object-in-hand status, subgoal completion, and other variables not fully observable from the current image. Let o_t denote the observation, including visual and proprioceptive inputs. In a partially observable Markov decision process, the sufficient statistic for optimal decision making is the belief state

$$b_t(z) = p(z_t = z \mid o_{1:t}, a_{1:t-1}). \quad (22)$$

The standard discrete-time Bayes filter updates the belief by combining a prediction step with an observation likelihood:

$$b_t(z_t) = \eta p(o_t \mid z_t) \sum_{z_{t-1}} p(z_t \mid z_{t-1}, a_{t-1}) b_{t-1}(z_{t-1}), \quad (23)$$

where η is a normalization constant.

In continuous time, if the latent Markov dynamics have generator Q , the prediction step over an elapsed interval Δt_t is

$$\bar{b}_t = \exp(Q \Delta t_t) b_{t-1}. \quad (24)$$

Thus, belief evolution depends on elapsed physical time, not only on the number of policy queries. This motivates memory updates whose retention is explicitly a function of Δt_t .

In TFP, we do not represent the full probabilistic belief b_t . Instead, the memory state $h_t \in \mathbb{R}^{d_h}$ is a learned belief embedding that summarizes task progress and interaction history.

B. LTC as First-Order Belief Relaxation

Assume that, at query t , the current observation and prior memory induce a candidate belief

$$\hat{h}_t = C_\theta(x_t, h_{t-1}), \quad (25)$$

where x_t is the compact observation representation. Consider the following continuous-time relaxation dynamics for each latent belief coordinate j :

$$\frac{dh_j(u)}{du} = -\frac{1}{\tau_{t,j}} (h_j(u) - \hat{h}_{t,j}), \quad u \in [t - \Delta t_t, t], \quad (26)$$

where $\tau_{t,j} > 0$ is the time constant for memory channel j . We assume that $\hat{h}_t$ and τ_t are held fixed over the interval between two policy queries.

a) *Proposition 1.*: The exact solution of Eq. (26) over elapsed time Δt_t is

$$h_{t,j} = \exp(-\Delta t_t / \tau_{t,j}) h_{t-1,j} + (1 - \exp(-\Delta t_t / \tau_{t,j})) \hat{h}_{t,j}. \quad (27)$$

Equivalently, in vector form,

$$h_t = k_t \odot h_{t-1} + (1 - k_t) \odot \hat{h}_t, \quad k_t = \exp(-\Delta t_t / \tau_t). \quad (28)$$

b) *Proof.*: For each dimension j , Eq. (26) is a linear first-order ODE:

$$\frac{dh_j(u)}{du} = -\frac{1}{\tau_{t,j}} h_j(u) + \frac{1}{\tau_{t,j}} \hat{h}_{t,j}.$$

Let the initial value at the beginning of the interval be $h_j(t - \Delta t_t) = h_{t-1,j}$. Solving the ODE gives

$$h_j(u) - \hat{h}_{t,j} = \exp\left(-\frac{u - (t - \Delta t_t)}{\tau_{t,j}}\right) (h_{t-1,j} - \hat{h}_{t,j}).$$

Setting $u = t$ yields

$$h_{t,j} - \hat{h}_{t,j} = \exp(-\Delta t_t / \tau_{t,j}) (h_{t-1,j} - \hat{h}_{t,j}).$$

Rearranging gives Eq. (27). Applying this coordinate-wise gives Eq. (28). $\square$

This shows that the LTC update is not an arbitrary interpolation rule: it is the exact discrete-time update of a continuous-time belief relaxation process. The candidate $\hat{h}_t$ acts as evidence induced by the current observation, the previous memory h_{t-1} acts as the retained prior belief, and the time-dependent retention k_t determines how strongly the prior persists over the elapsed interval.

C. Elapsed-Time Consistency of Exponential Retention

The exponential form in Eq. (28) is also the natural form required by elapsed-time consistency. Consider a scalar memory channel whose retention over elapsed time Δ is $k(\Delta)$. If the same interval is split into two sub-intervals, Δ_1 and Δ_2 , then applying retention over Δ_1 and then over Δ_2 should be equivalent to applying it once over $\Delta_1 + \Delta_2$. Therefore, a time-consistent retention function must satisfy

$$k(\Delta_1 + \Delta_2) = k(\Delta_1)k(\Delta_2). \quad (29)$$

a) *Proposition 2.*: Assume $k : [0, \infty) \rightarrow (0, 1]$ is continuous, satisfies $k(0) = 1$, and obeys Eq. (29). Then

$$k(\Delta) = \exp(-\lambda\Delta) \quad (30)$$

for some $\lambda \geq 0$. Equivalently,

$$k(\Delta) = \exp(-\Delta/\tau)$$

for $\tau = 1/\lambda$, with $\tau = \infty$ when $\lambda = 0$.

b) *Proof.*: Since $k(\Delta) > 0$, define $r(\Delta) = \log k(\Delta)$. Taking logarithms of Eq. (29) gives

$$r(\Delta_1 + \Delta_2) = r(\Delta_1) + r(\Delta_2).$$

Because k is continuous, r is continuous. The continuous solutions to Cauchy's additive functional equation on $[0, \infty)$ are linear:

$$r(\Delta) = c\Delta.$$

Since $k(\Delta) \in (0, 1]$, we have $r(\Delta) \leq 0$, so $c = -\lambda$ for some $\lambda \geq 0$. Thus,

$$k(\Delta) = \exp(r(\Delta)) = \exp(-\lambda\Delta).$$

$\square$

Thus, exponential retention is not an arbitrary design choice. It is the unique continuous scalar retention rule that is consistent with elapsed-time composition. LTC parameterizes this retention with an input-dependent time constant,

$$\tau_t = \tau_\theta(x_t, h_{t-1}),$$

allowing each memory channel to adapt its effective time scale to the current belief and evidence.

D. Belief Stabilization Is Not Memory Collapse

The LTC update also explains why low update gain does not necessarily indicate memory degeneration. Suppose that within a manipulation stage, the relevant latent belief is approximately constant:

$$z_t = z.$$

Assume that the candidate belief produced from the current observation is noisy:

$$\hat{h}_t = z + \epsilon_t, \quad \mathbb{E}[\epsilon_t] = 0, \quad \text{Var}(\epsilon_t) = \sigma^2. \quad (31)$$

For a scalar memory channel with constant retention k , the update is

$$h_t = kh_{t-1} + (1 - k)(z + \epsilon_t). \quad (32)$$

a) *Proposition 3.*: Under the assumptions above and with $h_0 = 0$, the mean and variance of h_t are

$$\mathbb{E}[h_t] = (1 - k^t)z, \quad (33)$$

and

$$\text{Var}(h_t) = (1 - k)^2 \sigma^2 \frac{1 - k^{2t}}{1 - k^2}. \quad (34)$$

For large t ,

$$\text{Var}(h_t) \approx \sigma^2 \frac{1 - k}{1 + k}. \quad (35)$$

b) *Proof.*: Unrolling Eq. (32) gives

$$h_t = k^t h_0 + (1 - k) \sum_{i=1}^t k^{t-i} (z + \epsilon_i).$$

With $h_0 = 0$,

$$h_t = (1 - k) \sum_{i=1}^t k^{t-i} z + (1 - k) \sum_{i=1}^t k^{t-i} \epsilon_i.$$

Since $\mathbb{E}[\epsilon_i] = 0$,

$$\mathbb{E}[h_t] = (1 - k) \sum_{i=1}^t k^{t-i} z = (1 - k)(1 + k + \dots + k^{t-1})z.$$

Using the geometric series identity,

$$1 + k + \dots + k^{t-1} = \frac{1 - k^t}{1 - k},$$

we obtain

$$\mathbb{E}[h_t] = (1 - k^t)z.$$

For the variance, only the noise term contributes:

$$\text{Var}(h_t) = \text{Var} \left[(1 - k) \sum_{i=1}^t k^{t-i} \epsilon_i \right].$$

Assuming the noise terms are independent with variance σ^2 ,

$$\text{Var}(h_t) = (1 - k)^2 \sum_{i=1}^t k^{2(t-i)} \sigma^2.$$

The remaining sum is

$$\sum_{i=1}^t k^{2(t-i)} = 1 + k^2 + \dots + k^{2(t-1)} = \frac{1 - k^{2t}}{1 - k^2}.$$

Thus,

$$\text{Var}(h_t) = (1 - k)^2 \sigma^2 \frac{1 - k^{2t}}{1 - k^2}.$$

For large t , $k^{2t} \rightarrow 0$ when $|k| < 1$, and since $1 - k^2 = (1 - k)(1 + k)$, we get

$$\text{Var}(h_t) \approx \sigma^2 \frac{1 - k}{1 + k}.$$

□

Equation (35) shows that when the candidate belief is noisy but the underlying task belief is stable, larger retention k reduces the variance of the memory state. Therefore, a high-retention regime can represent belief stabilization rather than collapse. In manipulation, this is desirable when slow variables such as task progress, target identity, or object-in-hand status should persist across occlusion, contact noise, and action chunks.

TABLE V: Hidden-state intervention under a fixed observation.

Metric	Value
Hidden conditions	25
Pairwise comparisons	300
Mean pairwise dist.	0.0442
Median pairwise dist.	0.0363
Max pairwise dist.	0.1303
Ref. distance, mean	0.0324
Ref. distance, max	0.1209
Adjacent dist., median	0.0108

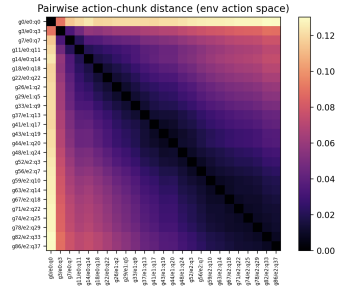


Fig. 5: Action-chunk variation caused by changing only the LTC hidden state under the same observation.

APPENDIX D

ADDITIONAL EXPERIMENTAL DETAILS

A. Same-Observation Hidden-State Intervention

To isolate the effect of the LTC memory from perception, we conduct a controlled hidden-state intervention. We select one query from a LIBERO rollout and fix its observation, robot state, and language instruction. We then evaluate the policy multiple times while replacing only the LTC hidden state h_t with hidden states collected from 25 different rollout timesteps. All other inputs are kept unchanged. For each hidden condition, the policy predicts a 10-step action chunk in the environment action space.

Given two predicted action chunks $A_i, A_j \in \mathbb{R}^{H \times d_a}$, we compute their normalized L2 distance:

$$D(i, j) = \frac{1}{H} \sum_{r=1}^H \|A_{i,r} - A_{j,r}\|_2.$$

We report pairwise distances across all hidden-state pairs, distances to a reference hidden state, and distances between temporally adjacent hidden states. Since the observation, robot state, and instruction are fixed, any action variation is caused by the memory intervention.

B. Failure Modes on MIKASA-Robo ShellGameTouch

MIKASA-Robo ShellGameTouch provides a useful diagnostic for testing whether a policy can preserve information that is no longer visible in the current observation. In our same-backbone evaluation, TFP achieves the success rate of 75.0%, indicating that the learned belief state provides useful memory under occlusion. However, the remaining failures also reveal an important boundary of the proposed method: ShellGameTouch requires a memory regime that is only partially aligned with the event-sensitive manipulation dynamics targeted by TFP.

a) *A complementary memory regime.*: TFP is designed to model continuous task-progress dynamics in manipulation. Its LTC belief update is most useful when the policy must decide when to preserve slow task variables and when to revise them around interaction events such as contact, grasp, release, placement, or subgoal completion. ShellGameTouch has a different structure. The decisive variable is a discrete hidden object-location binding: which visually identical mug

hides the ball. Once the ball is occluded, the current observation provides little corrective evidence about this latent identity-location association. The policy must therefore write the correct binding early and retrieve it at the final touch decision. This is closer to object-centric episodic memory or key-value binding than to continuously evolving manipulation-stage dynamics.

b) Sparse opportunities for belief correction.: In many manipulation tasks, the belief can be refined as new contact, motion, object-state, or subgoal-completion evidence appears. For example, after a grasp, release, or placement event, the observation gives new evidence about task progress, and an event-sensitive memory update can revise the internal state accordingly. In ShellGameTouch, by contrast, the crucial information is available only before and during occlusion. After the mugs become visually ambiguous, later observations mainly support execution rather than correction of the hidden ball-location belief. As a result, a weak or incorrect initial memory write is difficult to repair through subsequent LTC updates.

c) Representation mismatch.: This failure mode can also be viewed through a representation-learning lens. Prior work on interpreting neural robot policies argues that useful policy representations often align with task-relevant factors of variation, such as skills, behaviors, or strategies [32]. Under this view, ShellGameTouch stresses a different dominant factor of variation from the one most directly targeted by TFP. The task-relevant factor in ShellGameTouch is a persistent categorical object-location variable, whereas the factor targeted by TFP is event-sensitive task-progress dynamics. TFP can still improve ShellGameTouch because its belief state preserves useful latent context, but the memory is not explicitly structured as a symbolic object map, slot memory, or key-value store over hidden cup identities.

d) Failure categories.: Qualitatively, we observe four main failure modes. First, the policy may fail to encode the correct ball–mug association before occlusion, especially when the relevant visual cue is brief or spatially small. Second, the memory may retain a coarse task-progress signal but not a sufficiently precise object-location binding, causing the final touch to target the wrong mug. Third, the retained belief may be correct, but the action decoder may fail to ground that belief into the correct end-effector target under visually similar mugs. Finally, some failures are low-level execution errors near the final target rather than clear memory errors. These categories suggest that ShellGameTouch requires both persistent memory and precise object-centric grounding.

e) Interpretation.: We therefore treat ShellGameTouch as a same-backbone diagnostic rather than a direct state-of-the-art comparison. The improvement over the reactive baselines shows that TFP provides useful memory under occlusion, but the absolute success rate reflects the mismatch between ShellGameTouch’s categorical hidden-location memory and TFP’s primary design target: continuous-time, event-sensitive manipulation belief dynamics. A natural future direction is to combine TFP with an object-centric memory module, such as a slot-based belief map or a retrieval mechanism for discrete

hidden-state bindings. Such a hybrid would allow the policy to retain symbolic object-location facts while still using LTC dynamics to update task-progress belief around manipulation events.

APPENDIX E

STATISTICAL RELIABILITY OF ROLLOUT RESULTS

Because several LIBERO splits are close to saturation, we report confidence intervals for reproduced rollout results. For standard LIBERO, each suite contains 10 tasks and each task is evaluated with 50 rollout trials, giving 500 rollouts per suite. For each success rate, we compute a Wilson 95% confidence interval from the number of successful rollouts. For method differences on standard LIBERO, we report approximate two-proportion 95% confidence intervals against the reproduced $\pi_{0.5}$ baseline. We report intervals only for our reproduced methods; published prior-method numbers in Table I are not necessarily evaluated under the same rollout protocol.

TABLE VI: Statistical reliability of reproduced LIBERO rollout results. Each LIBERO suite contains 10 tasks, and each task is evaluated with 50 rollout trials, giving 500 rollouts per suite. We report success counts, Wilson 95% confidence intervals for success rates, and approximate 95% confidence intervals for absolute improvements over $\pi_{0.5}$.

Benchmark	Split	$\pi_{0.5}$	TFP	Δ vs. $\pi_{0.5}$
LIBERO	Spatial	494/500 = 98.8% [97.4, 99.4]	498/500 = 99.6% [98.6, 99.9]	+0.8 pp [-0.3, +1.9]
LIBERO	Object	490/500 = 98.0% [96.4, 98.9]	495/500 = 99.0% [97.7, 99.6]	+1.0 pp [-0.5, +2.5]
LIBERO	Goal	491/500 = 98.2% [96.6, 99.1]	497/500 = 99.4% [98.3, 99.8]	+1.2 pp [-0.1, +2.5]
LIBERO	Long	462/500 = 92.4% [89.7, 94.4]	485/500 = 97.0% [95.1, 98.2]	+4.6 pp [+1.8, +7.4]
LIBERO	Avg.	1937/2000 = 96.85% [96.0, 97.5]	1975/2000 = 98.75% [98.2, 99.2]	+1.90 pp [+0.99, +2.81]

TABLE VII: Statistical reliability of real-world Galaxea A1 results. We report Wilson 95% confidence intervals for success rates over 20 trials per method and task.

Task	Method	Success rate
Object swap	$\pi_{0.5}$	3/20 = 15.0% [5.2, 36.0]
Object swap	TFP	15/20 = 75.0% [53.1, 88.8]
Counting pick-place	$\pi_{0.5}$	8/20 = 40.0% [21.9, 61.3]
Counting pick-place	TFP	18/20 = 90.0% [69.9, 97.2]